EXPLORA CANDELARIA

Manual del sistema

Arquitectura · Instalación · Datos · Operación

Edición 1.0 · 12 de octubre de 2026
Fuente funcional: Rama-Daniela
Documento publicado en main

Guía técnica para mantenimiento, desarrollo y operación. Incluye diccionario de datos y evidencia de verificación local.

Contenido

1. Presentación, alcance y versión · 3

2. Arquitectura y tecnologías · 4

3. Organización del proyecto · 5

4. Requisitos de ejecución · 6

5. Instalación en un entorno nuevo · 7

6. Variables de configuración · 8

7. Arranque y procesos operativos · 9

8. Módulos, rutas y autorización · 10

9. Modelo de datos y relaciones · 11

10. Reservas, disponibilidad y cálculo · 12

11. Automatización, archivos e integraciones · 13

12. Despliegue y actualización · 14

13. Copias de seguridad y mantenimiento · 15

14. Pruebas y simulación · 16

15. Limitaciones y diagnóstico · 17

Anexo A. Diccionario de datos · 18

Anexo B. Fuentes y control de cambios · 30

1. Presentación, alcance y versión

Este manual técnico describe la estructura y operación de Explora Candelaria, un sistema web de información turística y reservas. Está dirigido a desarrolladores, administradores de infraestructura y responsables de soporte. Complementa el manual de usuario de turistas, administradores y emprendedores.

Fecha: 12 de octubre de 2026. Fuente funcional: Rama-Daniela, commit 3943f13d45944d979c9d7375506ee2e5e3c0128e. Los documentos se publican en main. Guardarlos en main no incorpora en esa rama el código de Rama-Daniela ni hace equivalentes sus dependencias o su esquema.

La documentación se basa en la inspección del código, los archivos de configuración, las migraciones y comprobaciones locales. Los procedimientos de despliegue y restauración son recomendaciones operativas; no se ejecutó un despliegue de producción, una recuperación real ni una prueba de carga certificada.

2. Arquitectura y tecnologías

La aplicación utiliza Laravel como servidor, Livewire para componentes interactivos, Blade para vistas y Eloquent para persistencia. Las rutas aplican autenticación y roles; controladores y componentes delegan operaciones a servicios de negocio. Vite construye los recursos estáticos. El navegador también carga recursos externos del mapa y de las vistas públicas.

Componente	Versión del lock / función
Laravel	12.53.0 · aplicación, rutas y persistencia
Livewire	4.2.1 · interacción de componentes
Flux	2.13.0 · componentes de interfaz
Fortify	1.35.0 · autenticación y recuperación
Spatie Permission	7.2.2 · roles y permisos
Dompdf	3.1.2 · reportes PDF
PhpSpreadsheet	5.7.0 · dependencia declarada; no implica que todos los reportes exporten Excel
Pest	4.4.1 · pruebas
Vite / Tailwind	Vite 7 y Tailwind 4 según package.json
Otras dependencias	FullCalendar 6, Chart.js 4, Axios y google/cloud-translate

Flujo de una solicitud: navegador → rutas y middleware → controlador o componente Livewire → servicio → modelo Eloquent → base de datos. Las vistas Blade presentan el resultado. Los archivos se guardan mediante Storage y los correos mediante Mail.

Capa	Responsabilidad
Presentación	resources/views, layouts de turista y paneles, componentes Livewire.
Aplicación	routes, controladores, acciones Fortify y proveedores.
Negocio	app/Services: reservas, inventario, servicios y emprendimientos.
Datos	app/Models, database/migrations y seeders.
Operación	config, storage, colas y routes/console.php.

3. Organización del proyecto

Directorio / archivo	Contenido
app/Livewire/Admin	Gestión administrativa y publicaciones.
app/Livewire/Emprendimiento	Servicios, reservas y reportes del establecimiento.
app/Livewire/Turista	Buscador, mapa, checkout e historial.
app/Livewire/Settings	Perfil, contraseña, apariencia, 2FA, datos comerciales y de pago.
app/Services	Operaciones y reglas de negocio.
app/Mail y app/Notifications	Mensajes de registro, recuperación y reservas.
app/Simulacion	Simulador de rendimiento y generación de reportes de simulación.
routes/web.php / settings.php	Rutas funcionales y configuración personal.
routes/console.php	Tarea de revisión de reservas cada minuto.
bootstrap/app.php	Configuración inicial y alias de middleware.
database/migrations	Evolución del esquema; fuente de referencia de la base.
resources / public	Código de recursos, vistas y archivos accesibles por HTTP.
storage	Archivos, registros, sesiones y cachés locales.
composer.lock / package-lock.json	Versiones resueltas de dependencias.
.github/workflows	Comprobaciones de formato y pruebas en CI.

4. Requisitos de ejecución

Se recomienda PHP 8.4 y Composer 2 para el desarrollo. composer.json declara PHP ^8.2, pero el conjunto bloqueado de desarrollo contiene dependencias que requieren PHP 8.3 o superior. CI usa PHP 8.4 y 8.5. Confirma siempre los requisitos con composer check-platform-reqs.

Node 22 es la referencia de CI; Node 24 estaba disponible en el entorno usado para elaborar los documentos. Vite 7 necesita una versión de Node compatible. Instala con npm ci para conservar el lockfile.

Extensiones: PDO y controlador de la base elegida; DOM/XML, mbstring, OpenSSL, Fileinfo, GD, ZIP y las demás indicadas por Composer. Para SQLite instala pdo_sqlite y sqlite3. MySQL o PostgreSQL necesitan sus respectivos controladores y una validación independiente; aquí se comprobó SQLite.

Necesitas escritura en storage y bootstrap/cache, espacio para dependencias y archivos subidos, y acceso a los repositorios y servicios externos utilizados. No se establece una capacidad de producción garantizada: dimensiona con carga representativa y monitoreo.

```
php -v
php -m
composer --version
node --version
npm --version
composer check-platform-reqs
```

5. Instalación en un entorno nuevo

Utiliza el checkout existente cuando ya esté preparado. Para una instalación nueva de la versión documentada, clona Rama-Daniela. Conserva los cambios locales y revisa el estado de Git antes de instalar. Los siguientes comandos son instrucciones de instalación, no una publicación automática.

```
git clone --branch Rama-Daniela --single-branch \
  https://github.com/jhinier/proyecto_reservas_turisticas.git
cd proyecto_reservas_turisticas
composer install --no-interaction --prefer-dist
npm ci
```

Crea .env a partir de .env.example únicamente si no existe. Configura la base, URL y correo. En una instalación inicial sin clave, genera APP_KEY; nunca regeneres una clave de una instalación con datos cifrados existentes.

```
php -r "file_exists('.env') || copy('.env.example', '.env');"
# Solo en una instalación nueva sin APP_KEY:
php artisan key:generate
# Crea el archivo SQLite configurado si usas SQLite.
php artisan migrate
php artisan storage:link
npm run build
php artisan migrate:status
php artisan route:list
```

Ejecuta las migraciones antes de cargar catálogos. TipoServicioSeeder contiene Guianza, Paquetes Turísticos, Alimentación, Hospedaje y Alquiler de Equipos. Existe TipoPublicacionSeeder para el catálogo de publicaciones. Revisa los seeders antes de ejecutarlos.

```
php artisan db:seed --class=TipoServicioSeeder
php artisan db:seed --class=TipoPublicacionSeeder
```

DatabaseSeeder llama a permisos, que crea roles, permisos y una cuenta administrativa con datos prefijados. No ejecutes este seeder indiscriminadamente en una base existente o producción. Revisa la cuenta inicial, establece credenciales privadas y evita repetir operaciones no idempotentes. Ninguna contraseña del seeder se reproduce en este manual.

Si vendor está versionado en un checkout distinto, una instalación normal puede modificar archivos protegidos. Comprueba git ls-files vendor y utiliza una copia aislada o un procedimiento aprobado de actualización. La rama funcional inspeccionada no versiona vendor. No cambies lockfiles para resolver una instalación sin investigar antes el requisito ausente.

6. Variables de configuración

Variables	Uso
APP_NAME, APP_ENV, APP_URL	Nombre, entorno y URL pública; enlaces de correos dependen de APP_URL.
APP_KEY	Clave de cifrado; conservarla y respaldarla de forma segura.
APP_DEBUG	true en desarrollo; false en producción.
DB_CONNECTION, DB_DATABASE	Motor y base. En SQLite, DB_DATABASE puede ser una ruta absoluta.
DB_HOST, DB_PORT, DB_USERNAME, DB_PASSWORD	Conexión a motores remotos cuando corresponda.
MAIL_MAILER, MAIL_HOST, MAIL_PORT	Transporte y servidor de correo.
MAIL_USERNAME, MAIL_PASSWORD	Autenticación del correo; valores privados fuera de Git.
MAIL_FROM_ADDRESS, MAIL_FROM_NAME	Remitente de los mensajes.
QUEUE_CONNECTION	Backend de colas; .env.example propone database.
CACHE_STORE, SESSION_DRIVER	Backend de caché y sesiones; ejemplo con database.
FILESYSTEM_DISK	Disco de almacenamiento predeterminado.
LOG_CHANNEL, LOG_LEVEL	Canal y nivel de registros.
AWS_*, REDIS_*	Solo si el despliegue utiliza esos proveedores.

Para desarrollo local, MAIL_MAILER=log registra el contenido de los mensajes y no entrega correo real. No compartas registros que contengan enlaces de verificación, datos personales o credenciales. La confirmación del registro pendiente se conserva en caché durante 30 minutos: el backend debe mantenerse disponible durante ese periodo.

No subas .env, auth.json, credenciales, claves privadas o bases de datos con información real. Los requisitos FLUX_USERNAME y FLUX_LICENSE_KEY aparecen en CI; su ausencia no demuestra por sí sola que los paquetes utilizados requieran licencia privada. Comprueba la operación real y solicita credenciales solo para repositorios privados que efectivamente las necesiten.

7. Arranque y procesos operativos

En desarrollo convencional, composer dev inicia servidor, escucha de colas y Vite. El programador de tareas requiere un proceso adicional. También puedes iniciar cada proceso en una terminal independiente para identificar fallos.

```
php artisan serve
php artisan queue:listen --tries=1
npm run dev
php artisan schedule:work
```

En el entorno de nube preparado, PHP está instalado dentro de /workspace/.cloud-tools. Activa su envoltura en cada shell. Laravel debe conservar las variables del PHP local al crear el servidor; por eso se usa --no-reload. Este ajuste es específico de ese entorno.

```
export PATH=/workspace/.cloud-tools/bin:$PATH
export COMPOSER_HOME=/workspace/.cloud-tools/composer-home
cd /workspace/manual-rama-daniela
php artisan serve --no-reload --host=127.0.0.1 --port=8010
```

routes/console.php registra procesarCancelacionesAutomaticas cada minuto. En producción, ejecuta schedule:run desde cron cada minuto y supervisa los trabajadores de cola. Evita iniciar dos programadores para la misma instancia sin definir cómo impedir ejecución duplicada.

```
* * * * * cd /ruta/al/proyecto && php artisan schedule:run >> /ruta/al/log/scheduler.
log 2>&1
```

No uses artisan serve ni el servidor de Vite como servidores de producción. Publica los recursos compilados, configura un servidor HTTP con PHP-FPM y usa public como raíz web. Muchos mensajes de reserva llaman Mail::send directamente; disponer de un worker no convierte esos envíos en asíncronos.

8. Módulos, rutas y autorización

Rol / módulo	Rutas representativas y controles
Público	/, /servicios, /mapa, /sitios, /actividades y /festividades.
Registro personalizado	POST /registro; /registro/confirmar/{token}; /registro/estado.
Turista	/turista/checkout y /turista/mis-reservas; auth y verified en el grupo; checkout verifica el rol turista.
Administrador	/admin/panel; /admin/emprendimientos/gestion; /admin/festividades/gestion; /admin/reportes. Role: superAdministrador o administrador_gad.
Publicaciones administrativas	/admin/admin/sitios y /admin/admin/actividades; prefijo duplicado existente en esta versión.
Emprendimiento	/emprendimiento/panel; /emprendimiento/mis-servicios/servicios; /emprendimiento/reservas; /emprendimiento/reportes. Role: emprendimiento y middleware emprendimiento.activo.
Configuración	/settings/profile, /settings/password, /settings/appearance, /settings/two-factor.
Configuración comercial	/settings/emprendimiento y /settings/emprendimiento/datos-pago; rol emprendimiento.
Salud	/up, registrada por bootstrap/app.php.

VerificarEmprendimientoActivo comprueba la asociación del usuario con un establecimiento y su estado. Si está inactivo, cierra la sesión y redirige al acceso. ReservaService limita consultas del turista por user_id y del emprendimiento mediante la relación de servicios; conserva estos filtros al extender funciones.

Fortify proporciona recuperación de contraseña, verificación y 2FA. Los límites configurados son cinco intentos por minuto para acceso y desafío 2FA. El acceso examina el rol; una cuenta sin rol puede ser rechazada. El registro personalizado y el registro estándar de Fortify coexisten como superficies distintas y deben probarse por separado.

9. Modelo de datos y relaciones

La base migrada contiene 33 tablas, incluidas las tablas técnicas de Laravel. Las fechas de cada servicio reservado viven en reserva_detalles, no en la cabecera reservas. La relación del establecimiento con la reserva se obtiene a través de los servicios y su categoría pivote.

Entidad	Responsabilidad y relación
users	Datos personales, acceso y verificación; se asocia a reservas y emprendimientos.
emprendimientos	Establecimiento y responsable user_id. Tiene categorías y cuentas bancarias.
tipo_servicios	Catálogo de categorías.
emprendimiento_tipo_servicios	Pivote establecimiento/categoría con estado.
servicios	Oferta; pertenece al pivote; precio y stock.
detalle_hospedajes / detalle_guianzas	Propiedades específicas: capacidad o personas.
detalle_alimentaciones / detalle_paquete_turistico	Datos de alimentación o del paquete.
imagen_servicios	Imágenes asociadas a un servicio.
reservas	Cabecera: turista, estado, total, auditoría y comprobante.
reserva_detalles	Líneas: servicio, fechas, hora, cantidad, personas, precio unitario y subtotal.
cuentas_bancarias	Banco, número y titular del emprendimiento.
publicaciones_turisticas	Cabecera de contenido turístico asociada a usuario y tipo.
sitios_turisticos / actividades_turisticas / festividades	Extensiones de una publicación.
actividades_festividades	Programación de una festividad.
imagen_publicaciones	Galería de publicaciones.
roles / permissions y pivotes	Autorización mediante Spatie.
sessions / cache / jobs y auxiliares	Infraestructura Laravel.

Relaciones principales: User 1:N Reserva; Reserva 1:N ReservaDetalle; Servicio 1:N ReservaDetalle; Emprendimiento 1:N EmprendimientoTipoServicio; TipoServicio 1:N EmprendimientoTipoServicio; EmprendimientoTipoServicio 1:N Servicio; Emprendimiento 1:N CuentaBancaria. Las cardinalidades describen las claves y modelos; verifica restricciones únicas antes de asumir una relación exclusiva de negocio.

users, emprendimientos y servicios utilizan borrado lógico. Las claves de varias tablas incluyen eliminación en cascada para borrado físico. Evalúa las consecuencias antes de purgar registros: borrar lógicamente, borrar físicamente y cancelar una reserva son operaciones distintas.

10. Reservas, disponibilidad y cálculo

La selección del turista se conserva en sesión mediante reserva_turista_carrito y reserva_turista_datos. Checkout verifica la cuenta y registra la solicitud mediante ReservaService. El flujo del emprendimiento permite buscar a un turista existente por cédula y crear la reserva desde el panel.

Origen / evento	Comportamiento implementado
Reserva del turista	Crea cabecera Pendiente y procesa las líneas del carrito.
Reserva del emprendimiento	Crea cabecera Confirmada.
Confirmación del establecimiento	Cambia la reserva a Confirmada y emite instrucciones por correo.
Comprobante del turista	Solo sobre Confirmada; almacena archivo y fecha; cambia a Pago en revisión.
Revisión del responsable	Puede marcar Completada según las acciones visibles.
Cancelación	Guarda estado y motivo/auditoría según el flujo.
Reagendamiento	Valida fechas y disponibilidad y actualiza los detalles.

InventarioService calcula disponibilidad diaria para las categorías aplicables. Su consulta de ocupación considera estados Confirmada y Pendiente. Pago en revisión y Completada no participan en esa consulta: es una limitación que debe revisarse antes de confiar en el control de cupos en producción.

ReservaService utiliza transacciones y lockForUpdate al procesar servicios. La eficacia de los bloqueos depende del motor: SQLite no ofrece los mismos bloqueos de fila que MySQL o PostgreSQL. No se verificó prevención de sobreventa bajo concurrencia real.

Categoría	Fórmula del subtotal en ReservaService
Paquete turístico	precio × cantidad.
Hospedaje	precio × personas × cantidad × días.
Guianza / Alquiler	precio × cantidad × días.
Alimentación y otras	precio × cantidad.

El número de días se calcula de forma inclusiva, con mínimo uno. El precio_unitario del detalle conserva el precio al registrar la reserva. El total suma los subtotales calculados. Estas fórmulas describen el código; confirma con negocio si un hospedaje debe cobrarse por días inclusivos o por noches.

La búsqueda pública exige tres días de anticipación. Los paquetes no pueden empezar en domingo ni lunes. La cancelación por turista exige más de 24 horas antes del primer detalle; se permite en Pendiente o Confirmada sin comprobante dentro del plazo previsto.

11. Automatización, archivos e integraciones

La tarea de cada minuto cancela Pendientes con 24 horas o más desde la creación; cancela Confirmadas sin comprobante con 24 horas o más desde la actualización; avisa al responsable en la ventana de 23 a 24 horas desde el comprobante y marca Completada al superar 24 horas en revisión. También se invoca procesamiento al abrir la gestión de reservas del emprendimiento.

Existe un aviso de interfaz que menciona 12 horas para aprobación automática, mientras ReservaService usa 24. No comunique un plazo garantizado hasta resolver la discrepancia. Una tarea fallida, un cron detenido o un correo con error pueden retrasar el procesamiento.

Las imágenes y comprobantes se guardan en el disco public bajo storage/app/public y se exponen con public/storage. Una ruta pública no protege por sí misma un comprobante bancario. Evalúa almacenamiento privado y descarga autorizada para documentos sensibles antes de producción.

Integración	Necesidad y comportamiento
Correo	Confirmación de registro, recuperación y avisos de reservas. MAIL_MAILER=log solo simula la entrega en registros.
PDF	Dompdf genera los reportes; el controlador del emprendimiento revisado devuelve PDF.
Mapa	Leaflet desde unpkg.com; capas de cartocdn.com, ArcGIS y OpenTopoMap.
Traducción pública	Script translate.google.com; la dependencia PHP cloud-translate declarada no prueba que exista una integración de servidor activa.
Estilos públicos	cdn.tailwindcss.com; su bloqueo deja algunas vistas sin estilos.
Fuentes	fonts.bunny.net en vistas con head compartido.

Mantén TLS y verificaciones de integridad de los gestores de paquetes. Añade únicamente los destinos necesarios a la política de red. Si el CDN no está disponible, debe corregirse el acceso o adoptarse una solución de recursos locales mediante un cambio de código separado.

12. Despliegue y actualización

Procedimiento recomendado: respaldar; preparar la versión en pruebas; instalar dependencias desde lockfiles; compilar assets; configurar el entorno; aplicar migraciones revisadas; publicar con raíz public; iniciar PHP-FPM, colas y programador; validar comportamiento. No se ejecutó este procedimiento en producción durante la elaboración.

```
composer install --no-dev --no-interaction --prefer-dist --optimize-autoloader
npm ci
npm run build
php artisan migrate --force
php artisan storage:link
php artisan config:cache
php artisan view:cache
php artisan queue:restart
php artisan schedule:list
```

Ejecuta estos comandos solo en el despliegue preparado y con autorización operativa. migrate --force aplica cambios reales. No regeneres APP_KEY. No ejecutes migrate:fresh en datos reales. Verifica el comportamiento después de cachear configuración; limpia cachés cuando cambies variables, rutas o vistas.

Comprueba /up, acceso, registro personalizado, recuperación, permisos, catálogo, reserva, comprobante y reporte. Una respuesta de salud no demuestra que correo, inventario o pagos funcionen. Supervisa la URL de APP_URL, HTTPS, sesiones, tamaño de carga y permisos de los directorios.

13. Copias de seguridad y mantenimiento

Respalda la base, storage/app con archivos cargados, configuración privada y APP_KEY mediante un canal seguro; conserva además el commit y lockfiles de la versión. Los documentos del repositorio no sustituyen un respaldo de datos. Las credenciales y respaldos personales no se almacenan en GitHub.

Para SQLite en uso, utiliza una copia consistente mediante la API de respaldo o la herramienta sqlite3, no una copia improvisada durante escrituras. Para motores remotos, emplea sus utilidades de respaldo y una estrategia de restauración propia. Define retención y realiza pruebas periódicas en un entorno aislado.

Orden recomendado de recuperación: preparar runtime y código compatible; restaurar base y archivos; recuperar la clave y configuración; comprobar permisos; iniciar servicios; verificar autenticación, archivos, reservas y tareas. No pruebes la restauración sobre la única copia de producción.

Controla logs, fallos de correo, trabajos fallidos, estado del cron, espacio en disco y errores de consultas. Si cambias tablas o reglas de reserva, actualiza este manual y el manual de usuario. Revisa enlaces públicos de archivos, límites de carga, roles y políticas de acceso.

```
php artisan queue:failed
php artisan schedule:list
php artisan migrate:status
php artisan optimize:clear
```

14. Pruebas y simulación

CI prepara PHP 8.4/8.5 y Node 22, instala dependencias, genera una clave de prueba, compila assets y ejecuta Pest. El script composer test incluye primero lint:check; si el formato falla, las pruebas pueden no llegar a ejecutarse. Para diagnosticar por separado, ejecuta comprobación de formato y pruebas directamente.

```
composer check-platform-reqs
php vendor/bin/pint --test
php artisan test
# Solo con una base aislada y datos de prueba:
php artisan simulacion:rendimiento --concurrency=3 --iterations=10 --mode=mixed
```

El simulador admite modos create, update y mixed; limita concurrencia a 100 e iteraciones a 1000 y escribe reportes en storage/simulacion_reportes. Sus umbrales están en SimulacionConfig. El nombre ISO 25023 en el comando no constituye una certificación; la simulación no sustituye una prueba HTTP de carga representativa.

Verificación local del 12 de octubre de 2026: composer check-platform-reqs pasó con PHP 8.4.26; las 22 migraciones figuraron aplicadas; route:list y schedule:list se ejecutaron. php artisan test ejecutó 52 pruebas: 43 aprobadas y 9 fallidas, con 142 aserciones. La suite no está aprobada por completo.

Grupo de fallos	Observación del resultado
Acceso y desafío 2FA	Usuarios de prueba sin rol son rechazados por la autenticación.
Recuperación	Tres pruebas esperan la notificación estándar ResetPassword que no fue observada.
Registro estándar	Binding de CreatesNewUsers no resoluble en ese flujo de prueba.
Dashboard	Prueba espera 200, pero el comportamiento observado devuelve redirección 302.
Configuración 2FA	Prueba busca texto inglés que no aparece en la respuesta.

Estos resultados corresponden a Rama-Daniela y reemplazan cualquier resultado antiguo de pruebas de main como evidencia para este manual. No se modificaron fuentes o tests para hacerlos pasar. No se probaron pagos reales, entrega SMTP, motores remotos, carga concurrente ni restauración de producción.

15. Limitaciones y diagnóstico

Situación	Diagnóstico / acción recomendada
Vista de sitio turístico falla	LandingController referencia turista.publicacion.detalle-sitio, pero el archivo inspeccionado está bajo livewire/turista/publicacion. Revisar la referencia.
Registro de usuarios administrativo	Ruta existente, menú no la muestra y formulario omite campos exigidos; no presentarlo como flujo completo.
Plazo del pago inconsistente	Aviso 12 h frente a procesamiento 24 h; unificar reglas y mensajes.
Disponibilidad tras cambio de estado	La ocupación considera solo Pendiente y Confirmada; evaluar revisión de pagos y completadas.
Recaudación	El indicador incluye Confirmadas y Completadas; no equivale necesariamente a cobros comprobados.
SMTP o cron falla	Revisar logs, configuración efectiva y proceso programado; no reenviar masivamente sin comprobar efectos.
Clases ausentes / extensiones	Validar autoload y check-platform-reqs; instalar requisitos sin ignorarlos.
CSS público ausente	Comprobar CDN Tailwind; el build de Vite por sí solo no resuelve esa dependencia pública.

Estas observaciones requieren trabajo de código u operación separado. La generación y publicación del manual no las corrige. Al reportar un incidente, aporta el commit, ruta, rol y pasos, con registros sanitizados sin credenciales ni datos personales.

Anexo A. Diccionario de datos

Diccionario obtenido del esquema SQLite con las migraciones aplicadas. Los tipos mostrados son los declarados por SQLite; no representan todos los tipos nativos de otros motores. La marca PK identifica columnas que participan en la clave primaria. NOT NULL procede de PRAGMA y debe interpretarse junto con PK: SQLite puede no marcar NOT NULL en una PK. Las reglas del formulario pueden ser más restrictivas que el esquema.

actividades_festividades

Columna	Tipo	Restricción
id	INTEGER	PK NOT NULL
publicacion_id	INTEGER	NOT NULL
nombre	varchar	NOT NULL
fecha	date	NOT NULL
hora	time	NOT NULL
lugar	varchar	NOT NULL
descripcion	varchar	NOT NULL
imagen	varchar	según PK / admite NULL
created_at	datetime	según PK / admite NULL
updated_at	datetime	según PK / admite NULL

Claves foráneas: publicacion_id → publicaciones_turisticas.id; ON DELETE CASCADE

actividades_turisticas

Columna	Tipo	Restricción
publicacion_id	INTEGER	PK NOT NULL
duracion_estimada	varchar	NOT NULL
dificultad	varchar	NOT NULL
recomendaciones	varchar	NOT NULL
created_at	datetime	según PK / admite NULL
updated_at	datetime	según PK / admite NULL

Claves foráneas: publicacion_id → publicaciones_turisticas.id; ON DELETE CASCADE

cache

Columna	Tipo	Restricción
key	varchar	PK NOT NULL
value	TEXT	NOT NULL
expiration	INTEGER	NOT NULL

cache_locks

Columna	Tipo	Restricción
key	varchar	PK NOT NULL
owner	varchar	NOT NULL
expiration	INTEGER	NOT NULL

cuentas_bancarias

Columna	Tipo	Restricción
id	INTEGER	PK NOT NULL
emprendimiento_id	INTEGER	NOT NULL
nombre_banco	varchar	NOT NULL
numero_cuenta	varchar	NOT NULL
titular	varchar	NOT NULL
created_at	datetime	según PK / admite NULL
updated_at	datetime	según PK / admite NULL
deleted_at	datetime	según PK / admite NULL

Claves foráneas: emprendimiento_id → emprendimientos.id; ON DELETE CASCADE

detalle_alimentaciones

Columna	Tipo	Restricción
servicio_id	INTEGER	PK NOT NULL
tipo_alimentacion	varchar	NOT NULL
lugar_alimentacion	varchar	NOT NULL
created_at	datetime	según PK / admite NULL
updated_at	datetime	según PK / admite NULL

Claves foráneas: servicio_id → servicios.id; ON DELETE CASCADE

detalle_guianzas

Columna	Tipo	Restricción
servicio_id	INTEGER	PK NOT NULL
numero_max_persona	INTEGER	NOT NULL
created_at	datetime	según PK / admite NULL
updated_at	datetime	según PK / admite NULL

Claves foráneas: servicio_id → servicios.id; ON DELETE CASCADE

detalle_hospedajes

Columna	Tipo	Restricción
servicio_id	INTEGER	PK NOT NULL
capacidad	INTEGER	NOT NULL
created_at	datetime	según PK / admite NULL
updated_at	datetime	según PK / admite NULL

Claves foráneas: servicio_id → servicios.id; ON DELETE CASCADE

detalle_paquete_turistico

Columna	Tipo	Restricción
servicio_id	INTEGER	PK NOT NULL
lugar_salida	varchar	NOT NULL
hora_salida	time	NOT NULL
servicios_incluidos	TEXT	NOT NULL
lugares_actividades	TEXT	NOT NULL
recomendaciones	TEXT	NOT NULL
documento	varchar	NOT NULL
mensaje_pago	varchar	según PK / admite NULL
created_at	datetime	según PK / admite NULL
updated_at	datetime	según PK / admite NULL
duracion_dias	INTEGER	NOT NULL

Claves foráneas: servicio_id → servicios.id; ON DELETE CASCADE

emprendimiento_tipo_servicios

Columna	Tipo	Restricción
id	INTEGER	PK NOT NULL
tipo_servicio_id	INTEGER	NOT NULL
emprendimiento_id	INTEGER	NOT NULL
created_at	datetime	según PK / admite NULL
updated_at	datetime	según PK / admite NULL
estado	tinyint(1)	NOT NULL

Claves foráneas: emprendimiento_id → emprendimientos.id; ON DELETE CASCADE; tipo_servicio_id → tipo_servicios.id; ON DELETE CASCADE

emprendimientos

Columna	Tipo	Restricción
id	INTEGER	PK NOT NULL
user_id	INTEGER	NOT NULL
nombre	varchar	NOT NULL
descripcion	varchar	NOT NULL
estado	tinyint(1)	NOT NULL
created_at	datetime	según PK / admite NULL
updated_at	datetime	según PK / admite NULL
imagen	varchar	según PK / admite NULL
deleted_at	datetime	según PK / admite NULL
enlaces	TEXT	según PK / admite NULL

Claves foráneas: user_id → users.id; ON DELETE CASCADE

failed_jobs

Columna	Tipo	Restricción
id	INTEGER	PK NOT NULL
uuid	varchar	NOT NULL
connection	TEXT	NOT NULL

Columna	Tipo	Restricción
queue	TEXT	NOT NULL
payload	TEXT	NOT NULL
exception	TEXT	NOT NULL
failed_at	datetime	NOT NULL

festividades

Columna	Tipo	Restricción
publicacion_id	INTEGER	PK NOT NULL
fecha_inicio	date	NOT NULL
fecha_fin	date	NOT NULL
created_at	datetime	según PK / admite NULL
updated_at	datetime	según PK / admite NULL

Claves foráneas: publicacion_id → publicaciones_turisticas.id; ON DELETE CASCADE

imagen_publicaciones

Columna	Tipo	Restricción
id	INTEGER	PK NOT NULL
publicacion_id	INTEGER	NOT NULL
imagen	varchar	NOT NULL
created_at	datetime	según PK / admite NULL
updated_at	datetime	según PK / admite NULL

Claves foráneas: publicacion_id → publicaciones_turisticas.id; ON DELETE CASCADE

imagen_servicios

Columna	Tipo	Restricción
id	INTEGER	PK NOT NULL
servicio_id	INTEGER	NOT NULL
imagen	varchar	NOT NULL
created_at	datetime	según PK / admite NULL
updated_at	datetime	según PK / admite NULL

Claves foráneas: servicio_id → servicios.id; ON DELETE CASCADE

job_batches

Columna	Tipo	Restricción
id	varchar	PK NOT NULL
name	varchar	NOT NULL
total_jobs	INTEGER	NOT NULL
pending_jobs	INTEGER	NOT NULL
failed_jobs	INTEGER	NOT NULL
failed_job_ids	TEXT	NOT NULL
options	TEXT	según PK / admite NULL
cancelled_at	INTEGER	según PK / admite NULL
created_at	INTEGER	NOT NULL
finished_at	INTEGER	según PK / admite NULL

jobs

Columna	Tipo	Restricción
id	INTEGER	PK NOT NULL
queue	varchar	NOT NULL
payload	TEXT	NOT NULL
attempts	INTEGER	NOT NULL
reserved_at	INTEGER	según PK / admite NULL
available_at	INTEGER	NOT NULL
created_at	INTEGER	NOT NULL

migrations

Columna	Tipo	Restricción
id	INTEGER	PK NOT NULL
migration	varchar	NOT NULL
batch	INTEGER	NOT NULL

model_has_permissions

Columna	Tipo	Restricción
permission_id	INTEGER	PK NOT NULL
model_type	varchar	PK NOT NULL
model_id	INTEGER	PK NOT NULL

Claves foráneas: permission_id → permissions.id; ON DELETE CASCADE

model_has_roles

Columna	Tipo	Restricción
role_id	INTEGER	PK NOT NULL
model_type	varchar	PK NOT NULL
model_id	INTEGER	PK NOT NULL

Claves foráneas: role_id → roles.id; ON DELETE CASCADE

password_reset_tokens

Columna	Tipo	Restricción
email	varchar	PK NOT NULL
token	varchar	NOT NULL
created_at	datetime	según PK / admite NULL

permissions

Columna	Tipo	Restricción
id	INTEGER	PK NOT NULL
name	varchar	NOT NULL
guard_name	varchar	NOT NULL
created_at	datetime	según PK / admite NULL
updated_at	datetime	según PK / admite NULL

publicaciones_turisticas

Columna	Tipo	Restricción
id	INTEGER	PK NOT NULL

Columna	Tipo	Restricción
user_id	INTEGER	NOT NULL
tipo_publicacion_id	INTEGER	NOT NULL
nombre	varchar	NOT NULL
descripcion	varchar	NOT NULL
created_at	datetime	según PK / admite NULL
updated_at	datetime	según PK / admite NULL

Claves foráneas: tipo_publicacion_id → tipo_publicaciones.id; ON DELETE CASCADE; user_id → users.id; ON DELETE CASCADE

reserva_detalles

Columna	Tipo	Restricción
id	INTEGER	PK NOT NULL
reserva_id	INTEGER	NOT NULL
servicio_id	INTEGER	NOT NULL
fecha_inicio	date	NOT NULL
fecha_fin	date	según PK / admite NULL
cantidad	INTEGER	NOT NULL
precio_unitario	numeric	NOT NULL
subtotal	numeric	NOT NULL
created_at	datetime	según PK / admite NULL
updated_at	datetime	según PK / admite NULL
hora_llegada	time	según PK / admite NULL
numero_personas	INTEGER	según PK / admite NULL

Claves foráneas: reserva_id → reservas.id; ON DELETE CASCADE; servicio_id → servicios.id; ON DELETE CASCADE

reservas

Columna	Tipo	Restricción
id	INTEGER	PK NOT NULL
user_id	INTEGER	NOT NULL

Columna	Tipo	Restricción
estado	varchar	NOT NULL
precio_total	numeric	NOT NULL
created_at	datetime	según PK / admite NULL
updated_at	datetime	según PK / admite NULL
reservada_por_rol	varchar	NOT NULL
cancelada_en	datetime	según PK / admite NULL
cancelada_por_rol	varchar	según PK / admite NULL
motivo_cancelacion	TEXT	según PK / admite NULL
comprobante_pago	varchar	según PK / admite NULL
fecha_subida_comprobante	datetime	según PK / admite NULL

Claves foráneas: user_id → users.id; ON DELETE CASCADE

role_has_permissions

Columna	Tipo	Restricción
permission_id	INTEGER	PK NOT NULL
role_id	INTEGER	PK NOT NULL

Claves foráneas: role_id → roles.id; ON DELETE CASCADE; permission_id → permissions.id; ON DELETE CASCADE

roles

Columna	Tipo	Restricción
id	INTEGER	PK NOT NULL
name	varchar	NOT NULL
guard_name	varchar	NOT NULL
created_at	datetime	según PK / admite NULL
updated_at	datetime	según PK / admite NULL

servicios

Columna	Tipo	Restricción
id	INTEGER	PK NOT NULL

Columna	Tipo	Restricción
emprendimiento_tipo_servicio_id	INTEGER	NOT NULL
nombre	varchar	NOT NULL
descripcion	TEXT	NOT NULL
precio	numeric	NOT NULL
stock	INTEGER	NOT NULL
created_at	datetime	según PK / admite NULL
updated_at	datetime	según PK / admite NULL
deleted_at	datetime	según PK / admite NULL

Claves foráneas: emprendimiento_tipo_servicio_id → emprendimiento_tipo_servicios.id; ON DELETE CASCADE

sessions

Columna	Tipo	Restricción
id	varchar	PK NOT NULL
user_id	INTEGER	según PK / admite NULL
ip_address	varchar	según PK / admite NULL
user_agent	TEXT	según PK / admite NULL
payload	TEXT	NOT NULL
last_activity	INTEGER	NOT NULL

sitios_turisticos

Columna	Tipo	Restricción
publicacion_id	INTEGER	PK NOT NULL
ubicacion	varchar	NOT NULL
created_at	datetime	según PK / admite NULL
updated_at	datetime	según PK / admite NULL

Claves foráneas: publicacion_id → publicaciones_turisticas.id; ON DELETE CASCADE

tipo_publicaciones

Columna	Tipo	Restricción
id	INTEGER	PK NOT NULL
nombre	varchar	NOT NULL
created_at	datetime	según PK / admite NULL
updated_at	datetime	según PK / admite NULL

tipo_servicios

Columna	Tipo	Restricción
id	INTEGER	PK NOT NULL
nombre	varchar	NOT NULL
created_at	datetime	según PK / admite NULL
updated_at	datetime	según PK / admite NULL

users

Columna	Tipo	Restricción
id	INTEGER	PK NOT NULL
name	varchar	NOT NULL
apellidos	varchar	NOT NULL
email	varchar	NOT NULL
cedula	varchar	NOT NULL
edad	INTEGER	NOT NULL
telefono	varchar	NOT NULL
email_verified_at	datetime	según PK / admite NULL
password	varchar	NOT NULL
remember_token	varchar	según PK / admite NULL
created_at	datetime	según PK / admite NULL
updated_at	datetime	según PK / admite NULL
two_factor_secret	TEXT	según PK / admite NULL
two_factor_recovery_codes	TEXT	según PK / admite NULL
two_factor_confirmed_at	datetime	según PK / admite NULL

Columna	Tipo	Restricción
deleted_at	datetime	según PK / admite NULL

Anexo B. Fuentes y control de cambios

Fuente: jhinier/proyecto_reservas_turisticas, Rama-Daniela, commit 3943f13d45944d979c9d7375506ee2e5e3c0128e. Inspección: composer.json/lock, package.json/lock, bootstrap/app.php, routes, config, modelos, servicios, componentes, migraciones, seeders y workflows. Esquema extraído de la base local independiente utilizada para documentación.

Edición 1.0 del manual del sistema, 12 de octubre de 2026. Publicación prevista en docs/manual-sistema de main. Complemento: ../manual-usuario. La estructura visual mantiene portada, secciones numeradas y una franja lateral, siguiendo el estilo utilizado en el manual de usuario.